1. Register User API:

1. API Endpoint:-

->Method: POST
->Full URL: http://localhost:8080/register

2. What Is the Use Case?

->This API is used to register a new user in the application.
->It receives the user's details and passes them to the service layer, which processes the registration and saves the user.

3. Request Body:

{

"id": 1,

"name": "Ravi",

"email": "ravi@gmail.com",

"password": "ravi123"

}

->The @RequestBody annotation converts the incoming JSON data into a Java User object.

4. Mandatory and Optional Fields

->**ID**: Depends on the ID configuration in the User entity.
->**Name**: User's name.
->**Email**: User's email address.
->**Password**: User's password.
->Exact mandatory and optional fields depend on the validation and registration logic in the User entity and UserService.

5. Responses

->Success: Returns the user's ID, name, and email.

{
"id": 1,
"name": "Ravi",
"email": "ravi@gmail.com"
}

->The password is not included because the controller returns a UserResponse containing only the ID, name, and email.
->If registration fails, an appropriate error response should be returned.
->Duplicate registration can be handled using UserAlreadyExsistException and the global exception handler.

2. Login API:

1. API Endpoint:-

->Method: POST mapping
->Full URL: http://localhost:8080/login

2. What Is the Use Case?

->This API verifies a user's login details.
->It receives the email and password and passes them to the UserService to check the credentials.

3. Request Body:-

```
{
  "email": "ravi@gmail.com",
  "password": "ravi123"
}
```

->The controller extracts the email and password from the Java User object and passes them to the login service method.

4. Mandatory and Optional Fields:-

Email: Required for checking the user's account.
Password: Required for verifying the login credentials.
Optional fields: No additional fields are required by the controller's login logic.

5. Responses:-
Success: Returns the user's ID, name, and email.

```
{
  "id": 1,
  "name": "Ravi",
  "email": "ravi@gmail.com"
}
```

->The password is not returned to the client.
->Incorrect credentials should be handled with an appropriate error response.
->In a real application, passwords should be securely hashed and verified.

3. Get All Products API:

1. API Endpoint:-

->Method: GET
->Full URL: http://localhost:8080/products

2. What Is the Use Case?

->This API retrieves all products stored in the database.
->It is useful for displaying available products to users.

->The controller calls the ProductService, which uses the repository's findAll() method to fetch the products.

3. Request Body:

->No request body is required. The client sends a GET request to retrieve the product list.

4. Mandatory and Optional Fields:

->Mandatory fields: None in the request body.
->Optional fields: None in the request body.

5. Responses:

->Success — 200 OK: Returns a list of products.


```
[
 {
  "id": 2,
  "name": "Lenovo Laptop",
  "price": 60000,
  "description": "Lenovo laptop"
 }
]
```


->If no products are available, the API can return an empty list:[]
->An empty list means the request succeeded, but no product records were found.

4. Create Product API:-

1. API Endpoint:
->Method: POST
->Full URL: http://localhost:8080/products

2. What Is the Use Case?

->This API adds a new product to the database.
->The client sends product details, and the controller passes them to the ProductService.
->The service uses the repository's save() method to store the product through Spring Data JPA.

3. Request Body:


```
{
 "id": 2,
 "name": "Lenovo Laptop",
 "price": 60000,
 "description": "Lenovo laptop"
}
```


->The @RequestBody annotation converts the JSON data into a Java Product object.

4. Mandatory and Optional Fields:
->ID: Must be supplied with the current configuration because automatic ID generation has not been enabled.
->Name: Product name.
->Price: Product price.
->Description: Product details.
->The current Product entity does not define validation rules that make name, price, or description mandatory.
5. Responses:
->Success — 200 OK: Returns the saved product details.

```
{
  "id": 2,
  "name": "Lenovo Laptop",
  "price": 60000,
  "description": "Lenovo laptop"
}
```

->If the database operation fails, the API should return an appropriate error response.

Global Exception Handling:

->Global exception handling provides a common place to handle exceptions across controllers.
->In this project, @RestControllerAdvice and @ExceptionHandler are used to handle UserAlreadyExsistException.
->When a duplicate-user exception occurs, the handler returns an error response containing the message, date, time, and status.
Example:

```
{
  "message": "User already exists",
  "time": "2026-10-26T09:30:00",
  "date": "2026-10-26",
  "status": "400 Bad Request"
}
```

->The date and time above are examples; actual values depend on when the exception occurs.

Conclusion:

->By creating these APIs, I learned how to accept JSON requests, process data through the service layer, perform database operations using Spring Data JPA, and return responses to the client.

->I also learned that request validation, secure password handling, and global exception handling are important for building reliable REST APIs.